

Gesture Mimic

Is Acceleration New Accuracy?

Acceleration $\equiv$ Reactivity $\equiv$ Task Accuracy

2 $\rightarrow$ 13–15 inf/s | MAE 6 $\rightarrow$ ~3 (usable limit ~3.5)

PAVS

Optimization Sequence — Throughput & Accuracy Journey

Step-by-step changes on SmoVLA + LeRobot (GPU / Strix)

#	Step	What changed	Impact
1	Diffusion steps ↓	Reduced denoising steps 30 → 2 (extra steps shown unnecessary)	2 → 4.5–5 inf/s
2	torch.compile	Rewrote LeRobot SmoVLA modules so the full flow forms a single graph → optimized GPU kernels	→ 8.5 inf/s
3	fp32 → fp16	Cast weights to fp16 <i>before</i> compile so the graph has no up/down-scale ops around each layer	→ 13–15 inf/s
4	RTC on (Real-Time Chunking)	Camera pipeline + model run on parallel threads sharing the GPU → reactive end-to-end pipeline	Reactivity ↑
5	chunk_size_threshold ↓	Next inference triggered after only 5 consumed actions → fresher images drive predictions	Reactivity ↑
6	Action smoothing	Post-processing removes jitter/shake in continuous-action output	Smoothness ↑
7	More data + fine-tune	Scaled training data and fine-tuned model	MAE 6 → ~3

Bottom line: ~7× throughput (2 → 13–15 inf/s) and ~50% lower MAE (6 → ~3, usable limit ~3.5) — model is now usable for live gesture mimic on Strix.

Why Throughput Matters

Speed is not just speed; it directly drives behavior quality

What each speed step bought us

- **Diffusion 30 → 2:** removed redundant compute — denoising had diminishing returns past 2 steps.
- **torch.compile:** fused operators into a single graph → optimized GPU kernels, no Python overhead per layer (but needed code rewrite).
- **fp16 (compile-aware):** halved memory & compute. Casting weights *before* compile is critical(tricky wrt rocm+torch.compile) — otherwise compile inserts upscale / downscale around every layer and the precision win is eaten by overhead.
- **RTC + lower `chunk_size_threshold`:** model and camera no longer wait on each other; new frames trigger inference every 5 actions instead of waiting for a full chunk.

Throughput ladder

Stage	inf/s
Baseline	2.0
+ Diffusion steps 30 → 2	4.5–5
+ torch.compile (graph)	8.5
+ fp16 (compile-aware)	13–15
+ RTC + chunk_threshold = 5	Reactive pipeline

Throughput → Accuracy Coupling

Faster model = more accurate, more responsive robot

Why a slow model hurts accuracy

- **Drops camera frames** → operator ↔ robot sync is lost.
- Forced to predict **far into the future** from stale observations — long-horizon predictions are inherently unreliable.
- Lag accumulates → robot mimics what the operator did *seconds ago*.

Why a fast model is more accurate

- Consumes **every fresh frame** from the camera stream.
- New predictions **overwrite** old ones, always grounded in current state → less guessing.
- Short, reliable horizons → tighter operator ↔ robot sync.

Key insight for leadership: throughput is not just speed — it directly raises **behavioral accuracy** by keeping predictions grounded in the latest observation.

Accuracy & Smoothness Gains

Data + fine-tune → MAE 6 → ~3

Stage	MAE
Past delivery model	6.0
+ Combined dataset & fine-tune	~3.0
Usable-limit threshold	~3.5

→ ~50% MAE reduction — now under the ~3.5 usability limit.

Smoothness — jitter removal

- Continuous action space + limited data → visible **shake/jitter** on the robot.
- Added lightweight **post-processing smoother** on the action stream.
- Smooth, deployable motion — **no retraining needed**.

Net result: real-time, smooth, accurate gesture-mimic stack on SmoIVLA + LeRobot.